

Definitive Plan (v1): LLM-Driven Stock & Options Trading Agent

Context

You want an "AI/LLM trader" that can (a) research markets using historical chart/price and other data, (b) generate and backtest algorithmic strategies, and (c) execute stock and options trades through a platform, with the best low-cost MCP integrations, buildable by a Replit agent, exportable to GitHub, and hostable on Vercel/VPS.

This plan reflects current (mid-2026) research and corrects one critical misconception up front, then gives a precise, low-cost stack and step-by-step build/run/test instructions.

The single most important correction (read this first)

TradingView is not an execution API. For retail, TradingView is a charting and signal tool. You cannot have an LLM "place trades through TradingView" via a clean API. The real-world pattern is: TradingView Pine Script **alert** -> **webhook** -> **bridge (e.g. TradersPost)** -> **broker**. That adds cost, latency, and a fragile hop. The better architecture for an LLM agent is to **talk to a broker API directly** (Alpaca), and optionally also ingest TradingView webhook signals as one input. This plan uses Alpaca as the execution layer and treats TradingView as optional signal input.

Reality-check commentary (please read before building)

- LLMs do **not** have an edge at price prediction. Their value here is orchestration, research synthesis, strategy generation, risk reasoning, and natural-language control -- **not** magic alpha. Treat the LLM as the "analyst + PM + risk officer" layer on top of deterministic, **backtested** strategies, not as an oracle.
- **Backtest results never guarantee live results** (overfitting, slippage, look-ahead bias, regime change). Options especially can lose 100% fast.
- **Therefore: paper-trade first, for weeks.** Hard risk limits in code, not in prompts.
- This is a build/engineering plan, not financial advice.

Assumptions (recommended defaults; each is an adjustable knob)

- **Architecture:** Standalone Python service (not built inside the eigent desktop app). Eigent can later act as an optional research "cockpit" via MCP.
- **Trade mode:** Paper-trading first, then live with hard limits + optional human-in-the-loop approval. No fully-autonomous live money until validated.
- **Broker / execution:** Alpaca (commission-free stocks + options, free paper account, clean REST API, official MCP server with ~65 tools).
- **Budget:** Ultra-low (<\$50/mo to build + paper-trade).

Why standalone (and why not just inside eigent)

The workspace repo `eigent` is a CAMEL-AI multi-agent desktop (Electron) app with a clean MCP/toolkit pattern. It's excellent for interactive, human-present workflows -- but a trader needs an always-on, scheduled, unattended loop (market-open jobs, position monitoring, stop management). A desktop app is the wrong runtime for that. So: build a standalone always-on service; if you want the eigent chat UI later, point it at the same Alpaca MCP server (hybrid).

Recommended Stack (definitive, low-cost)

Layer	Choice	Why / Cost
Language/ runtime	Python 3.11	Ecosystem for quant + LLM agents
Agent framework	LangGraph (mirror the open-source TradingAgents design)	Battle-tested multi-agent pattern for trading
LLM	Claude via Anthropic API. Haiku for loops, Sonnet for trader/risk, Opus for deep research. Prompt caching + batching.	Best cost/quality balance
Broker / execution	Alpaca (stocks + options, paper + live)	Commission-free, free paper, official MCP server
Execution MCP	alpaca-hq/alpaca-mcp-server (official, v2, ~65 tools)	Place/replace orders, positions, option chains in natural language
Market data	Alpaca Market Data (bundled); Polygon.io Free->Starter (\$0-29/mo) for deeper history/options	Cheapest viable
Backtesting	VectorBT (fast research) + Backtesting.py (validation); QuantLib for greeks; NautilusTrader later for parity	Separate discovery from execution realism
Scheduler	APScheduler or host-native cron/worker	Market-hours jobs
Storage	SQLite (start) -> Postgres (scale)	Audit log
Build env	Replit	Per request
Source	GitHub	Per request
Host (always-on)	Railway hobby or \$5-6/mo VPS (Hetzner/DO)	Always-on worker required
Host (NOT primary)	Vercel -- stateless dashboard only, not the trading loop	Serverless times out, no scheduler

Hosting verdict

Vercel cannot run the trading loop (ephemeral, time-limited functions, no persistent scheduler). Use Railway or a VPS for the always-on agent. Vercel is fine for a thin read-only dashboard or to receive TradingView webhooks and forward them.

Target Architecture

Data (Alpaca/Polygon) and optional TradingView webhook feed into a LangGraph agent graph (Analyst -> Bull/Bear -> Trader -> Risk), guarded by deterministic risk guardrails (max position, daily loss kill-switch, per-trade size, market-hours-only, paper/live switch), executing via the Alpaca MCP server (paper -> live), with a SQLite/Postgres audit log. Hard risk guardrails live in deterministic code the agent cannot bypass.

MCP Setups & Integrations

1. Alpaca MCP server (execution + account + data) -- primary. Clone alpaca-hq/alpaca-mcp-server, set ALPACA_API_KEY/SECRET, ALPACA_PAPER_TRADE=true.
2. Market-data / news MCP (optional): Polygon or Financial-Datasets MCP + a web-search MCP.
3. Reuse pattern from eigent if hybrid: add Alpaca MCP to ~/.eigent/mcp.json.

Repository Layout

A standalone Python project (ai-trader/) with src/ split into data, agents, strategies, backtest, risk, execution, scheduler, webhook, main; plus tests/ (test_risk.py critical), README, .env.example, mcp.json, Dockerfile, docker-compose.yml, optional dashboard/.

Step-by-Step: How a Replit Agent Builds It

1. Scaffold Python 3.11 project with uv.
2. Install deps: langgraph, langchain-anthropic, langchain-mcp-adapters, mcp, alpaca-py, vectorbt, backtesting, pandas, numpy, apscheduler, fastapi, uvicorn, pydantic, python-dotenv, pytest.
3. Secrets in Replit Secrets (never commit).
4. Data layer -> 5. Strategies -> 6. Backtester -> 7. RiskManager (tested) ->
5. Agent graph (LangGraph) -> 9. Execution via Alpaca MCP (paper only) ->
6. Scheduler -> 11. Optional webhook -> 12. Tests green -> 13. Docs.

Export to GitHub

Connect GitHub in Replit, create private repo, commit, push. Ensure .env is git-ignored, no secrets committed (secret-scan first). Add a GitHub Action running pytest on push.

Dry Runs / Test Runs

- Tier 0 Unit (no network): pytest green; risk guardrails proven.
- Tier 1 Backtest accuracy: 3-5 yrs history in VectorBT, cross-check in Backtesting.py, add slippage/ commission, check look-ahead bias.
- Tier 2 Paper dry run: full loop vs Alpaca paper; run weeks; compare vs backtest.
- Tier 3 Live, gated: tiny size, low limits, human-in-the-loop, kill-switch reachable.

Deploy / Install / Run

Local: clone, cp .env, uv sync, pytest, backtest, paper loop. Railway: deploy from GitHub, add Worker service, env vars, restart-on-crash (~\$5/mo). VPS: docker compose up -d, restart: unless-stopped. Vercel: dashboard only.

Cost Table (monthly)

Ultra-low ~\$5-40/mo (Alpaca \$0 + free/cheap data + cached Claude + \$5 host). Moderate ~\$80-200/mo (Polygon paid + Sonnet + bigger host).

Open questions / knobs

1. Standalone vs eigent vs hybrid? 2. Paper vs live-approval vs autonomous?
2. Broker Alpaca vs Tradier vs IBKR vs TradingView-bridge? 4. Budget tier?
3. Equities vs options emphasis?

Sources

- Alpaca MCP: github.com/alpacahq/alpaca-mcp-server; alpaca.markets/mcp-server
- Alpaca options: docs.alpaca.markets/us/docs/options-trading
- TradingView webhooks: [tradingview.com support 43000529348](https://tradingview.com/support/43000529348); traderspost.io

- TradingAgents: github.com/TauricResearch/TradingAgents; arxiv.org/pdf/2412.20138
- Market data: polygon.io/options
- Backtesting: autotradelab.com; bullalert.ai
- Hosting: vercel.com kb serverless timeouts; nandann.com python-hosting; docs.railway.com